

بررسی الگوریتم‌های فرا ابتکاری تنظیم‌کننده پارامترهای ماشین بولتزمن

محمد رحیمی*

۱- محمد رحیمی و rahimi_m72@noor.iust.ac.ir

چکیده

یادگیری ماشین رویکرد استقرایی است که از داده‌های آموزش دیده برای تولید مدل استفاده می‌کند. این مدل برای تشخیص تصویر، تشخیص گفتار، پردازش زبان طبیعی و ... مفید است. شبکه‌ی عصبی یکی از مدل‌های یادگیری ماشین است که به طور گسترده به عنوان یک مدل یادگیری ماشین مورد استفاده قرار می‌گیرد. شبکه‌های عصبی به دو دسته‌ی تک‌لایه و چند لایه تقسیم می‌شوند. یادگیری عمیق تکنیکی است که مشکلات شبکه‌های عصبی چندلایه را برطرف کرده است. تکنیک‌های یادگیری عمیق به چهار دسته‌ی شبکه‌های عصبی پیچشی، ماشین‌های رمزگذار، خودرمزگذارهای تنک و ماشین‌های بولتزمن تقسیم می‌شوند. یکی از محدودیت‌هایی که در شبکه‌های عصبی وجود دارد تنظیم تعداد زیادی از پارامترها است که می‌تواند به مسئله‌ی غیرقابل حل تبدیل شود. در این مقاله «تنظیم دقیق پارامترهای ماشین‌های بولتزمن با رویکردهای فرا ابتکاری» به عنوان یک مسئله‌ی بهینه‌سازی مورد بررسی قرار گرفته است.

کلمات کلیدی: شبکه‌های عصبی، ماشین‌های بولتزمن، تنظیم پارامتر، الگوریتم‌های فرا ابتکاری[†]

۱. مقدمه

یادگیری ماشین رویکرد استقرایی است که از داده‌های آموزش دیده برای تولید مدل استفاده می‌کند. این مدل برای تشخیص تصویر، تشخیص گفتار، پردازش زبان طبیعی و ... مفید است. موفقیت یادگیری ماشین به این وابسته است که چگونه فرآیند تعمیم پیاده‌سازی شود. کیفیت تعمیم به داده‌های آموزشی بی‌طرف وابسته است. چنانچه مدل بیش از حد به داده‌های آموزشی تکیه کند مشکل بیش‌برازش رخ می‌دهد. مدل‌های بیش‌برازش شده، نتایج ضعیفی را برای داده‌های واقعی ایجاد می‌کنند. در واقع بیش‌برازش یکی از عواملی است که کارایی تعمیم‌سازی را کاهش می‌دهد. تنظیم و اعتبارسنجی مدل رویکردهایی هستند که برای حل مشکل بیش‌برازش مورد استفاده قرار می‌گیرند. تنظیم مدل یک شیوه‌ی عددی است که

[†] metaheuristic

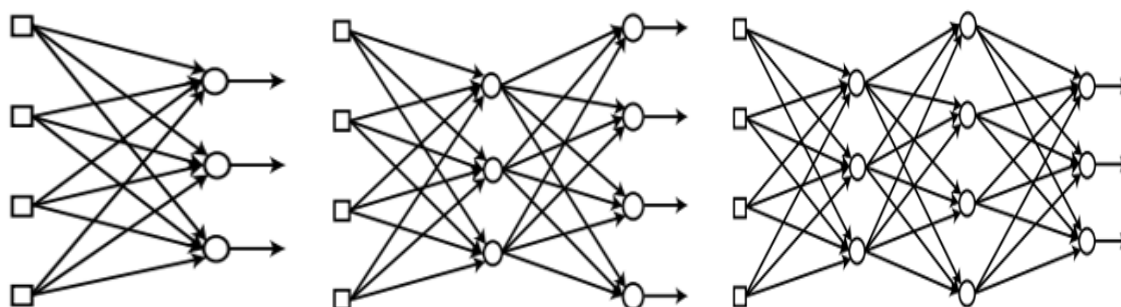
ساده‌ترین مدل ممکن را ارائه می‌دهد در مقابل اعتبارسنجی سعی می‌کند نشانه‌هایی که باعث پدیده‌ی بیش‌برازش شوند را شناسایی کرده و برای جلوگیری از آن‌ها اقدام کند. یادگیری ماشین با توجه به داده‌های آموزشی به سه دسته‌ی یادگیری با نظارت، یادگیری بدون نظارت و یادگیری تقویتی تقسیم می‌شود [1].

شبکه‌ی عصبی یکی از مدل‌های یادگیری ماشین می‌باشد که به طور گسترده به عنوان یک مدل یادگیری ماشین مورد استفاده قرار می‌گیرد. بیشتر شبکه‌های عصبی با لایه‌ای از گره‌ها ساخته می‌شوند. برای شبکه‌ی عصبی لایه‌ای، سیگنال‌ها از طریق لایه‌ی ورودی وارد می‌شوند، از لایه‌ی مخفی عبور می‌کنند و از لایه‌ی خروجی خارج می‌شوند. یادگیری با نظارت برای شبکه‌ی عصبی فرآیندی را پیاده‌سازی می‌کند که وزن‌ها را تنظیم کرده و خطای بین خروجی صحیح و خروجی شبکه‌ی عصبی را کاهش دهد. به طور عمده سه روش محاسبه‌ی خطا وجود دارد: کاهش شیب تصادفی، دسته و دسته‌ای کوچک. روش مورد استفاده برای تنظیم وزن‌ها قانون یادگیری نامیده می‌شود. قانون دلتا نماینده‌ی قانون یادگیری است که فرمول آن بسته به تابع فعال‌ساز متفاوت است.

$$\delta_i = \varphi'(v_i)e_i$$

$$w_{ij} \leftarrow w_{ij} + \alpha \delta_i x_j$$
(1)

قانون دلتا یک روش تکرارشونده است که به تدریج به راه‌حل دست پیدا می‌کند. بنابراین شبکه می‌بایست مکرراً با داده‌های آموزشی، آموزش ببیند تا خطا به حد مطلوبی کاهش یابد. شبکه‌ی عصبی تک لایه فقط برای انواع خاصی از مسائل قابل اجرا است. بنابراین موارد استفاده‌ی این شبکه بسیار محدود است. شبکه‌ی عصبی چند لایه برای غلبه بر محدودیت‌های ضروری شبکه‌ی عصبی تک لایه توسعه داده شده است [2]. شکل شماره‌ی ۱ از چپ به راست شبکه‌ی عصبی تک‌لایه، چند لایه‌ی کم‌عمق و چند لایه‌ی عمیق را نشان می‌دهد.



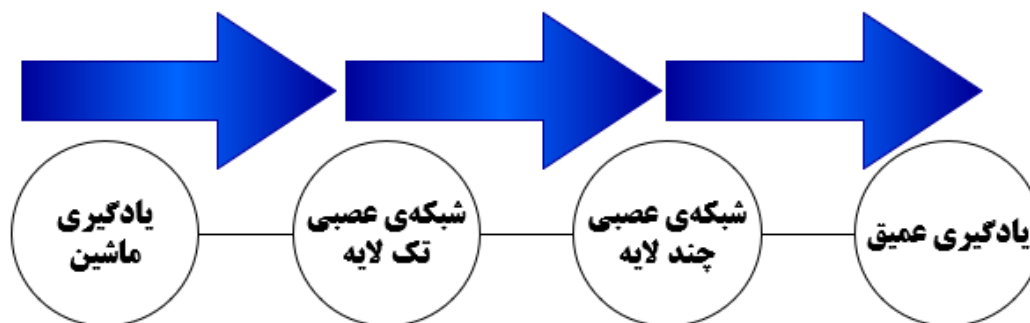
شکل ۱: شبکه‌های عصبی تک‌لایه، چند لایه‌ی کم‌عمق و چند لایه‌ی عمیق

شبکه‌ی عصبی چند لایه نمی‌تواند با استفاده از قانون دلتا آموزش ببیند. برای آموزش می‌بایست از الگوریتم پس‌انتشار استفاده شود که تحت عنوان قوانین یادگیری در یادگیری عمیق استفاده می‌شود. **الگوریتم پس انتشار** خطای لایه‌ی پنهان را با انتشار خطای خروجی از لایه‌ی خروجی به سمت عقب محاسبه می‌کند. پس از محاسبه‌ی خطا، وزن هر لایه تنظیم می‌شود. شبکه‌ی عصبی تک لایه فقط برای مسائلی مناسب است که به صورت خطی قابل تفکیک باشند در حالیکه بیشتر مسائل عملی به صورت خطی قابل تفکیک نیستند. بنابراین از شبکه‌ی عصبی چند لایه استفاده می‌شود. شبکه‌ی عصبی چند لایه توانایی مدل‌سازی مسائلی را دارد که به صورت خطی قابل تفکیک نیستند. توابع هزینه و فعال‌ساز، توابعی هستند که در شبکه‌ی عصبی چند لایه استفاده می‌شود. اندازه‌ی خطای شبکه‌ی عصبی توسط تابع هزینه محاسبه می‌شود. تابع آنتروپی متقابل یکی از پرکاربردترین توابع فعال‌سازی است که در شبکه‌ی عصبی چند لایه استفاده می‌شود [3].

برای **طبقه‌بندی شبکه‌ی عصبی**، انتخاب تعداد گره‌های خروجی و تابع فعال‌ساز معمولاً به دسته‌بندی باینری (دو کلاس) یا طبقه‌بندی چند کلاسه (سه یا بیشتر) بستگی دارد. برای طبقه‌بندی باینری، شبکه‌ی عصبی با یک گره‌ی خروجی منفرد و تابع فعال‌ساز سیگموئید ساخته می‌شود. خروجی صحیح داده‌های آموزشی به مقادیر حداقل و حداکثر تابع فعال‌ساز

تبدیل می‌شود. تابع هزینه قانون یادگیری از آنتروپی متقابل استفاده می‌کند. برای طبقه‌بندی چند کلاس، شبکه‌ی عصبی به اندازه‌ی تعداد کلاس‌ها، گره‌ی خروجی شامل می‌شود. تابع بیشینه‌ی هموار به عنوان تابع فعال‌ساز گره‌های خروجی استفاده می‌شود. با استفاده از روش رمزگذاری وان هات خروجی صحیح داده‌های آموزشی به بردار تبدیل می‌شود. تابع هزینه‌ی قانون یادگیری از آنتروپی متقابل استفاده می‌کند [4].

یادگیری عمیق به طور ساده یک تکنیک یادگیری ماشین است که از شبکه‌های عصبی عمیق استفاده می‌کند. آموزش لایه‌های پنهان عمیق‌تر سخت بود و باعث افت عملکرد آن می‌شد. یادگیری عمیق این مشکل را حل کرد. یادگیری عمیق سه نقطه ضعف شبکه‌ی عصبی عمیق را پوشش داد. این سه نقطه ضعف عبارتند از: محوشدگی گرادین، بیش‌برازش و بار محاسباتی. مشکل محوشدگی گرادین به طور عمده با استفاده از تابع فعال‌ساز ReLU و قانون یادگیری مبتنی بر آنتروپی متقابل بهبود یافت. استفاده از روش کاهش شیب نیز مفید است. مشکل بیش‌برازش با استفاده از حذف تصادفی یا نظم‌دهنده حل شد. همچنین مشکل محاسبات سنگین شبکه‌ی عصبی تا حد زیادی به وسیله‌ی GPU و الگوریتم‌های مختلف بهبود یافت [5]. مراحل شکل‌گیری یادگیری عمیق را می‌توان در شکل ۲ مشاهده نمود.



شکل ۲: مراحل شکل‌گیری یادگیری عمیق

در بخش دوم مقاله، تکنیک‌های یادگیری عمیق با محوریت بینایی کامپیوتر طبقه‌بندی شده است. به طوریکه یادگیری عمیق، به چهار دسته‌ی شبکه‌های عصبی پیچشی، ماشین‌های رمزگذار، خودرمزگذارهای تنک و ماشین‌های بولتزمن تقسیم می‌شوند.

شبکه‌ی عصبی پیچشی* یکی از مهمترین روش‌های یادگیری عمیق است که در دهه‌های ۱۹۸۰ و ۱۹۹۰ توسعه یافته است [6]. این شبکه‌ها تصویر ورودی را دریافت می‌کنند و به هر ویژگی تصویر، میزان اهمیتی (وزن‌های قابل یادگیری و بایاس) تخصیص می‌دهند که آن‌ها را قادر به متمایزسازی از یکدیگر می‌کند.

خود رمزگذارهای عمیق اولین بار توسط هینتون و همکارانش ارائه شدند [7] و در مطالعات زیادی از جمله [8]، [9] و [10] استفاده شده‌اند. این شبکه‌ها برای رمزگذاری بهینه‌ی عملیات یادگیری مورد استفاده قرار می‌گیرند.

کدگذاری تنک[†] نوع دیگری از انواع کدگذاری‌های عصبی است که برحسب موضع و محل قرارگیری نورون‌ها در برابر کدگذاری متراکم قرار دارد.

ماشین‌های بولتزمن محدودشونده[‡] [11, 12] مدل‌های احتمالی هستند که برای مدل‌سازی توزیع داده‌های ورودی (لایه پنهان) از یک لایه باینری پنهان استفاده می‌کنند که به آن‌ها واحدهای پنهان نیز می‌گویند.

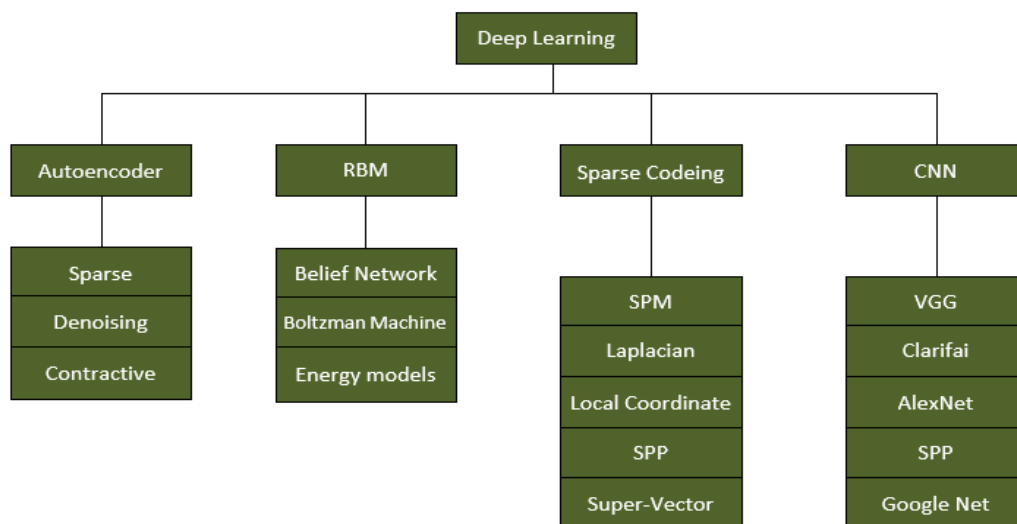
* Convolution Neural Network

[†]Sparse coding

[‡] Restricted Boltzmann Machines

یکی از محدودیت‌هایی که در شبکه‌های عصبی وجود دارد تنظیم تعداد زیادی از پارامترها است که می‌تواند به یک مسئله‌ی غیرقابل حل تبدیل شود. بخش سوم مقاله، مسئله‌ی «تنظیم دقیق پارامترهای ماشین‌های بولتزمن محدود شونده با الگوریتم‌های فرا ابتکاری» را به عنوان مسئله‌ی بهینه‌سازی مطرح می‌کند و الگوریتم‌های فرا ابتکاری استفاده شده در این زمینه مورد بررسی قرار می‌گیرد. این الگوریتم‌ها عبارتند از: جستجوی هارمونیک بهبود یافته (IHS)، الگوریتم بهینه‌سازی ازدحام ذرات وزن دار ساکن تطبیقی (AIWPSO)، جستجوی فاخته (CS)، الگوریتم کرم شب‌تاب (FA)، الگوریتم بهینه‌سازی جستجوی بازگشتی (BSA)، الگوریتم تکاملی دیفرانسیل تطبیقی (JADE) و تکامل دیفرانسیل بر اساس یادگیری ماتریس کوواریانس و الگوریتم تنظیم پارامترهای توزیع دو حالت (CoBiDE).

پاپا و همکارانش [13]، روزا و همکارانش [14] و پاسوس و همکارانش [15] از جمله اولین کسانی هستند که بهینه‌سازی مبتنی بر الگوریتم‌های فرا ابتکاری را در زمینه‌ی تنظیم دقیق پارامترهای ماشین‌های بولتزمن محدود شونده، ماشین‌های باور عمیق و ماشین‌های بولتزمن محدود شونده بی‌نهایت ارائه دادند و با استفاده از برخی کتابخانه‌های بهینه‌سازی شناخته شده، نتایج دقیق‌تری را نسبت به دیگران به دست آوردند. مزیت استفاده از این تکنیک‌ها آن است که به محاسبه‌ی مشتق صدها پارامتر نیاز ندارد البته این راه حل برای فضاهایی با ابعاد بالا توصیه نمی‌شود. در نهایت کارایی این الگوریتم‌ها مورد ارزیابی قرار می‌گیرد.



شکل ۳: برخی از انواع و مهمترین کاربردهای شبکه‌ی عصبی

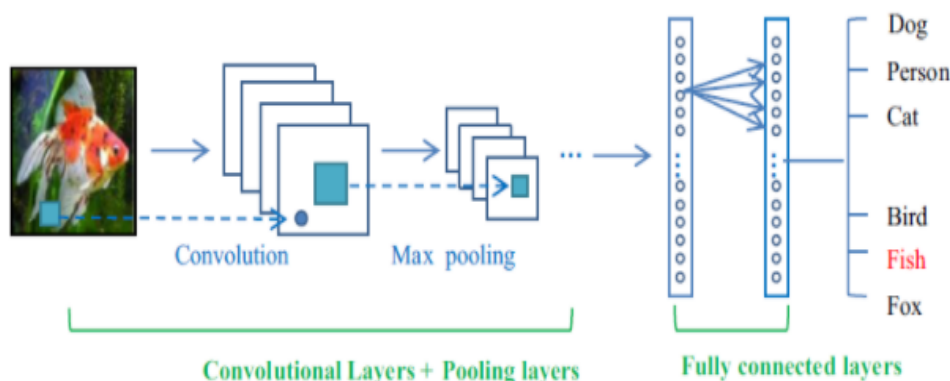
۲. مدل‌های مختلف یادگیری عمیق

طی سال‌های اخیر، یادگیری عمیق بصورت گسترده در حوزه بینایی کامپیوتر مورد مطالعه قرار گرفته است و به همین دلیل، تعداد زیادی از روش‌های مرتبط با آن بوجود آمده است. بطور کلی، این روش‌ها بر اساس روش پایه‌ای که از آن مشتق شده‌اند به ۴ دسته مختلف تقسیم می‌شوند که عبارتند از: یادگیری عمیق مبتنی بر پیچشی، یادگیری عمیق مبتنی بر ماشین‌های بولتزمن محدود شونده، یادگیری عمیق مبتنی بر خود رمزگذار، یادگیری عمیق مبتنی بر کدگذاری تنک. دسته‌بندی تکنیک‌های یادگیری عمیق به همراه برخی از کارهای انجام شده در این حوزه را می‌توان به طور خلاصه در شکل شماره‌ی ۳ مشاهده نمود.

۲-۱. شبکه‌ی عصبی پیچشی

شبکه‌ی عصبی پیچشی برای حل مسائل بینایی ماشین از قبیل تشخیص اشیاء، قطعه‌بندی و طبقه‌بندی تصاویر استفاده می‌شود. برخی از معماری‌های شبکه‌ی عصبی پیچشی [16] GoogleNet، [17] ResNet و [18] AlexNet هستند. معماری‌های پیچشی از سه لایه‌ی اصلی تشکیل شده‌اند: لایه پیچش، لایه پولینگ* و لایه تماماً متصل. شکل شماره‌ی ۴ معماری لایه‌بندی شده از شبکه‌ی عصبی پیچشی را برای دسته‌بندی تصاویر نشان می‌دهد [19].

آموزش شبکه‌های عصبی پیچشی دو مرحله دارد: مرحله تغذیه‌ی روبه‌جلو و مرحله پس‌انتشار. در مرحله اول تصویر ورودی به شبکه داده می‌شود و عملیات ضرب نقطه‌ای بین ورودی و وزن هر نورون انجام می‌شود و در نهایت عملیات پیچشی برای هر لایه اعمال می‌شود. سپس خروجی شبکه محاسبه می‌شود. در این جا به منظور تنظیم وزن نورون‌ها یا آموزش شبکه از حاصل خروجی برای محاسبه میزان خطای شبکه استفاده می‌شود. به این صورت که خروجی شبکه را با پاسخ صحیح مقایسه شده و میزان خطا از طریق تابع خطا محاسبه می‌شود. در گام بعدی بر اساس میزان خطای محاسبه شده، مرحله پس‌انتشار آغاز می‌شود. در این مرحله گرادیانت هر پارامتر با توجه به قاعده زنجیره‌ای محاسبه می‌شود و تمامی وزن‌ها با توجه به تأثیری که بر خطای ایجاد شده در شبکه دارند بروزرسانی می‌شوند. با تکرار تعداد مناسبی از این مراحل، آموزش شبکه پایان می‌یابد.



شکل ۴: معماری شبکه‌ی پیچشی

۲-۲. شبکه‌ی خودرمزگذار

شبکه‌ی خودرمزگذار برای استخراج ویژگی‌های داده‌ها و کاهش ابعادشان استفاده می‌شود. خود رمزگذارها نوع خاصی از شبکه‌های عصبی هستند که برای رمزگذاری بهینه‌ی عملیات یادگیری مورد استفاده قرار می‌گیرد [20]. این شبکه‌ها دارای لایه‌ی ورودی، لایه‌ی پنهان و لایه‌ی خروجی هستند که در ساده‌ترین حالت خود رمزگذار شامل یک رمزگذار و رمزگشا به همراه یک لایه‌ی پنهان است. خودرمزگذار سعی می‌کند که ورودی خود را بازسازی کند بنابراین ابعاد بردار خروجی با بردار ورودی برابر است یعنی تعداد نورون‌های موجود در لایه‌ی ورودی و خروجی با یکدیگر برابر هستند. شکل شماره ۵ معماری کلی شبکه‌ی عصبی خودرمزگذار را نشان می‌دهد [10].

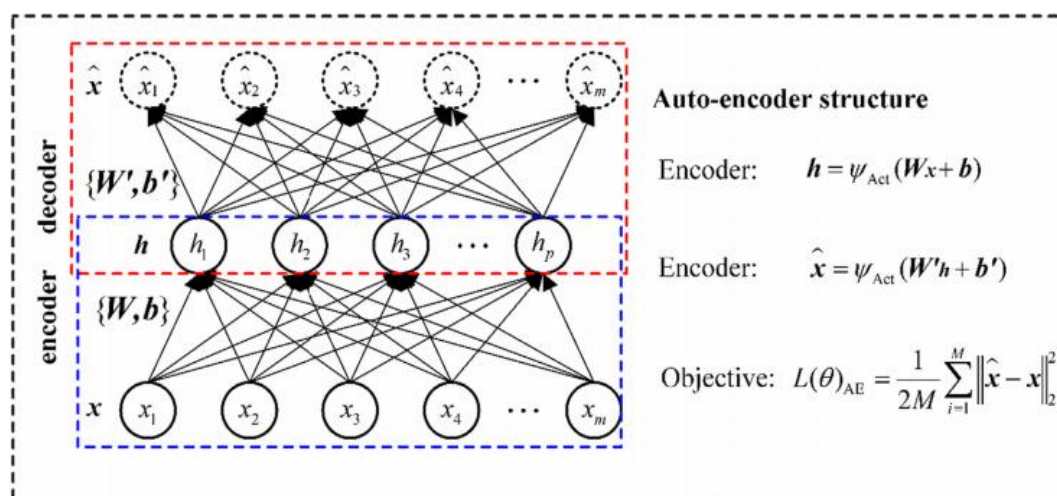
خودرمزگذارها شبکه را با حداقل کردن خطای بازسازی آموزش می‌دهند. معمولاً تعداد نورون‌ها در لایه‌ی پنهان از تعداد آن‌ها در لایه‌ی ورودی و خروجی کمتر است زیرا لایه‌ی پنهان داده‌ها را در ابعاد پایین‌تر نمایش می‌دهد. هر چه تعداد لایه‌های پنهان بیشتر باشد از ابعاد داده‌ها، بیشتر کاسته می‌شود. پس از انجام عملیات آموزش، شبکه رمزگشا حذف شده و خروجی میانی‌ترین لایه پنهان، به عنوان ویژگی‌های استخراج شده در نظر گرفته می‌شود. در صورتی که در لایه‌ی میانی

* Pooling

تنها از توابع خطی استفاده شود یا اینکه از یک لایه سگموئید استفاده شود عملکرد شبکه مطابق با تحلیل مولف‌های اصلی است [21].

خود رمزگذارهای خلوت^{*}، گونه‌ای از خود رمزگذارها هستند که بدنبال استخراج ویژگی‌های خلوت از داده‌های خام هستند. خلوت بودن نمایش‌ها می‌تواند از طریق جریمه کردن بایاس واحدهای مخفی [22,23] و یا به طور مستقیم با جریمه‌ی خروجی مقادیر واحد مخفی [23] بدست بیایند.

خود رمزگذار برطرف‌کننده‌ی نویز[†] که به اختصار با DAE نمایش می‌دهند، مدلی است که به منظور افزایش قدرت خود رمزگذارها توسط وینسنت و همکارانش [24, 25] ارائه شد. در این مدل ساختار توزیع ورودی‌ها دریافت می‌شود و ورودی صحیح از نسخه‌ی خراب بازیابی می‌شود. بدین صورت که اندکی نویز به سلول‌های ورودی اضافه و این شبکه، مجبور می‌شود که خروجی را از یک ورودی نویزی، بسازد و با این کار ویژگی‌های متداول بیشتری انتخاب می‌شود.



شکل ۵: معماری شبکه‌ی عصبی خودرمزگذار

خود رمزگذار قراردادی[‡] که به اختصار با CAE نمایش می‌دهند دنباله‌ی خود رمزگذارهای برطرف‌کننده‌ی نویز هستند که توسط ریفای و همکارانش ارائه شد [26]. با وجود آنکه DAE با تزریق نویز به مجموعه‌ی آموزشی باعث قدرتمند شدن عملیات نگاشت می‌شود خود رمزگذار قراردادی، با اضافه کردن جریمه‌ی قراردادی تحلیلی، عملیات نگاشت را بهبود می‌دهد.

۲-۳. کدگذاری خلوت

کدگذاری تنک یکی از انواع کدگذاری‌های عصبی است که برحسب موضع و محل قرارگیری نورون‌ها در برابر کدگذاری متراکم قرار دارد. به طوریکه در هر لحظه، تعدادی کمی از نورون‌ها فعال و بقیه خاموش هستند. در این مدل یک کلاس از الگوریتم‌ها برای بازنمایی مختصر داده‌های بدون برچسب انتخاب می‌شود. همچنین توابع پایه برای آموزش ویژگی‌های سطح بالای داده‌ها استفاده می‌شوند. این روش مزیت‌های زیر را دارد [27, 28, 29]:

➤ قادر به بازسازی بهتر توصیف‌گر با استفاده از چندین پایه و دریافت ارتباطات بین توصیف‌گرهای مشابهی است که دارای پایه‌های مشترکی هستند.

➤ تنک بودن کدگذاری خلوت اجازه می‌دهد تا نمایش، خصوصیت‌های برجسته داده ورودی را دریافت کند.

* Autoencoder Sparse

† Autoencoder Denoising

‡ Autoencoder Contractive

➤ این روش همسو با سیستم بینایی بیولوژیکی است که استدلال می‌کند ویژگی‌های خلوت سیگنال‌ها برای یادگیری مفید هستند.

➤ الگوها با ویژگی‌های پراکنده از لحاظ خطی بیشتر قابل جداسازی هستند.

➤ مطالعه آمار تصاویر نشان می‌دهد که قطعات تصویر سیگنال‌های پراکنده هستند.

۱-۳-۲. الگوریتم‌های توسعه یافته با رویکرد کدگذاری تنک

برخی از الگوریتم‌های توسعه یافته با رویکرد کدگذاری تنک عبارتند از:

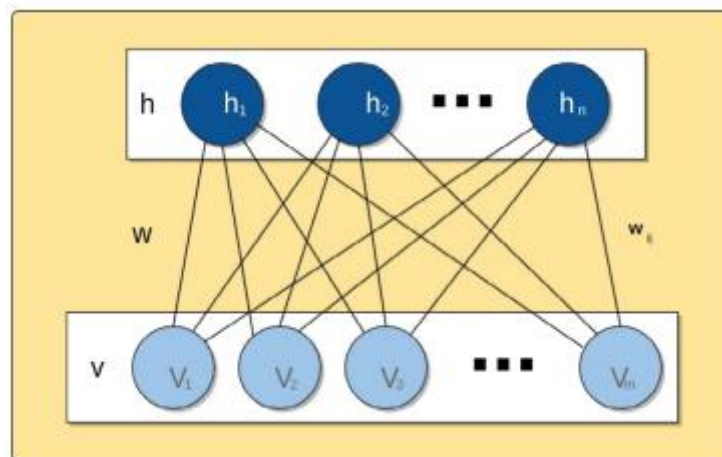
- ✓ **SPM**: در این روش از کمی‌سازی بردار (VQ) برای نمایش تصاویر استفاده می‌شود [30].
- ✓ **ScSPM**: در این روش از کدگذاری تنک به همراه تجمیع حداکثری فضای چند مقیاسی استفاده می‌شود [31]. یکی از مشکلات ScSPM این است که این روش به صورت جداگانه با ویژگی‌های محلی برخورد می‌کند و به این ترتیب وابستگی بین آن‌ها نادیده گرفته می‌شود که باعث حساسیت بیش از حد به واریانس ویژگی می‌شود بنابراین کدگذاری تنک ممکن است برای ویژگی‌های مشابه نیز به شدت تغییر کند [32].
- ✓ **LSC**: به منظور برطرف‌سازی مشکل ScSPM گو و همکارانش در مقاله‌ای [33] روش کدگذاری تنک لاپلاسی را ارائه کردند که در آن ویژگی‌های مشابه به مراکز دسته‌هایی منتسب می‌شود که بطور بهینه انتخاب شوند و تضمین می‌شود مراکز دسته‌های انتخابی مشابه یکدیگر هستند. با افزودن قید حفظ محلی بودن به اهداف کدگذاری تنک، LSC قادر به حفظ وابستگی دو طرفه در روشی کدگذاری تنک است.
- ✓ **HLSC**: بعد از ارائه LSC گو و همکارانش مدل کدگذاری تنک لاپلاسی هاپیرگراف را ارائه دادند که حاصل توسعه‌ی الگوریتم LSC بود. در این مدل شباهت‌ها در میان نمونه‌ها توسط یک هاپیرگراف تعریف می‌شود [34].
- ✓ **LCC**: این الگوریتم با توجه به خاصیت محلی بودن سعی در بهبود دادن الگوریتم ScSPM دارد. یو و همکارانش [35] نشان دادند که خاصیت محلی بودن پراکندگی را بهبود می‌دهد بنابراین کدگذاری تنک زمانی سودمند است که کدها محلی باشند.
- ✓ **LLC**: با توجه به آنکه محاسبات LLC زمان‌گیر است جین‌جونگ وانگ و همکارانش [36] به منظور تسریع فرآیند یادگیری، الگوریتم کدینگ خطی محدود شده‌ی محلی را ارائه دادند که به اختصار LLC نامیده می‌شود.
- ✓ **SVC**: کدگذاری بردار عالی (Super Vector Coding) که به اختصار SVC نامیده می‌شود بسط ساده‌ای از کمی‌سازی بردار (VQ) است که با گسترش این بردار، در جهات مماس محلی بدست می‌آید و به همین دلیل روش کدگذاری یکنواخت‌تری ایجاد می‌کند. این روش تنک بودن کدگذاری را تضمین می‌کند [37].
- ✓ **ASGD**: میانگین نزولی شیب تصادفی که به اختصار با ASGD نمایش می‌دهند با الگوریتم‌های LLC و SVC ترکیب شده است تا مقیاس دسته‌بندی تصاویر در مجموعه داده‌های عظیم افزایش یابد [38].
- ✓ **SSC**: کدگذاری تنک نرم که به اختصار با SSC نمایش می‌دهند توسط بالاسوبرامانیان و همکارانش ارائه شد. این روش با ترکیب شباهت همسایگی و اطلاعات موقتی در کدگذاری تنک توانست به کدهایی دست یابد که بجای نمونه انفرادی یک همسایگی را نمایش دهد و خطای مربع میانگین بازسازی کمتری داشته باشد [39].
- ✓ **DeepSC**: هو و همکارانش [40] چارچوب جدیدی از یادگیری‌های ویژگی بدون سرپرست به نام کدگذاری تنک عمیق ارائه کردند که به اختصار DeepSC نمایش می‌دهند. در این روش کدگذاری تنک با استفاده از یک معماری چند لایه توسعه می‌یابد.

۲-۴. ماشین بولتزمن محدود شده

ماشین بولتزمن محدود شده که به اختصار با RBM نمایش داده می‌شود، شبکه‌ی عصبی مولد تصادفی است که در سال ۱۹۸۶ توسط هینتون ارائه شد [41]. چنین مدل‌هایی برای مقابله با مشکلات مربوط به تصاویر [42]، متن [43]، تشخیص محتوای مخرب [44] [45] و تشخیص بیماری‌ها [46, 47] استفاده می‌شود. علاوه بر این، RBM ها در ساخت برخی از معماری‌های یادگیری عمیق از جمله شبکه‌ی باور عمیق [48] و ماشین بولتزمن عمیق [49] مورد استفاده قرار می‌گیرند که تفاوت اصلی آن‌ها مربوط به تعاملات بین لایه‌های RBM است.

۲-۴-۱. ماشین بولتزمن محدود شده

ماشین‌های بولتزمن محدود شده دارای دو لایه‌ی پنهان و مشخص هستند. تفاوت لایه‌ی مشخص در ماشین بولتزمن محدود شده و لایه‌ی ورودی شبکه‌ی عصبی معمولی آن است که نورون‌های لایه‌ی مشخص دارای بایاس هستند. همچنین RBM ها برای مدلسازی توزیع داده‌های ورودی از لایه‌ی باینری پنهان استفاده می‌کنند که به آنها واحدهای پنهان نیز می‌گویند [11, 12]. در ماشین‌های RBM تنها نورون‌های لایه‌ی مشخص و پنهان با یکدیگر ارتباط دارند و هیچ ارتباطی بین نورون‌های هر لایه برقرار نیست از این‌رو واژه‌ی محدود شده به ماشین بولتزمن اطلاق می‌شود. شکل شماره‌ی ۶ معماری ماشین بولتزمن محدود شده را نشان می‌دهد [50]:



شکل ۶: معماری RBM

هدف RBM یادگیری توزیع داده‌هاست. بدین منظور لازم است پارامترهای تابع انرژی به دقت تنظیم شوند تا مقدار این تابع برای داده‌هایی که شبیه به داده‌های آموزشی هستند بیشینه و برای داده‌های غیر مشابه کمینه شود. با فرض آنکه v و h به ترتیب مخفف لایه‌های مشخص و پنهان باشند معادله‌ی ۲ تابع انرژی این مدل را نشان می‌دهد [50]:

$$E(v, h) = - \sum_{i=1}^m a_i v_i - \sum_{j=1}^m b_j h_j - \sum_{i=1}^m \sum_{j=1}^n v_i h_j w_{ij} \quad (2)$$

در این معادله a و b به ترتیب مخفف بایاس واحد پنهان و مشخص است. از آنجا که RBM گراف دو بخشی است، فعال‌سازی هر دو واحد مشخص و پنهان به طور متقابل مستقل هستند و بنابراین منجر به احتمالات شرطی زیر می‌شود [50]:

$$p(v|h) = \prod_{i=1}^m p(v_i|h) \quad (3)$$

$$p(h|v) = \prod_{j=1}^n p(h_j|v) \quad (4)$$

$$p(v_i = 1|h) = \phi \left(\sum_{j=1}^n w_{ij} h_j + a_i \right) \quad (5)$$

$$p(h_j = 1|v) = \phi \left(\sum_{i=1}^m w_{ij} v_i + b_j \right) \quad (6)$$

در معادله‌ی شماره‌ی ۶ تابع سیگموئید $\phi(0)$ را نشان می‌دهد. فرض کنید پارامترهای ماشین بولتزمن محدودشونده $\theta = (W, a, b)$ و از طریق الگوریتم آموزشی قابل یادگیری باشند. هدف آن است که پارامترهای RBM به گونه‌ای آموزش ببینند که احتمال مشاهده شدن نمونه‌های آموزشی حداکثر شود. معادله‌ی شماره‌ی ۷ این مفهوم را نشان می‌دهد [50]:

$$\arg \max_{v \in V} \prod_{v \in V} P(v) \quad (7)$$

در معادله شماره‌ی ۷، $P(V)$ احتمال مشاهده شدن نمونه‌ها است. به جهت آنکه مقدار معادله برای نمونه‌های آموزشی بیشینه شود مشتق آن نسبت به پارامترهای W, a و b محاسبه می‌شود [50].

$$w^{t+1} = w^t + \eta (P(h|v)v^T - P(\tilde{h}|\tilde{v})\tilde{v}^T) + \phi = w^t + \Delta w^t \quad (8)$$

$$a^{t+1} = a^t + \eta (v - \tilde{v}) + \alpha \Delta a^{t-1} = a^t + \Delta a^t \quad (9)$$

$$b^{t+1} = b^t + \eta (P(h|v) - P(\tilde{h}|\tilde{v})) + \alpha \Delta b^{t-1} = b^t + \Delta b^t \quad (10)$$

پارامترهای α و η به ترتیب مومنتم و نرخ یادگیری مدل را نشان می‌دهند. برای بدست آوردن $P(\tilde{h}|\tilde{v})$ و $\tilde{v}$ از تکنیک واگرایی متقابل استفاده می‌شود [11] که عملیات نمونه‌گیری گیبس را از داده‌های آموزشی به عنوان واحدهای قابل مشاهده، انجام می‌دهند. به طور خلاصه معادلات ۷، ۸ و ۹ از گرادیان نزولی برای بهینه‌سازی الگوریتم استفاده می‌کند. همچنین در معادله‌ی شماره‌ی ۷، ϕ برای کنترل مقادیر ماتریس W در هنگام فرآیند همگرایی استفاده می‌شود [50].

$$\phi = -\lambda W^t + \alpha \Delta W^{t-1} \quad (11)$$

λ فروپاشی وزن را نشان می‌دهد.

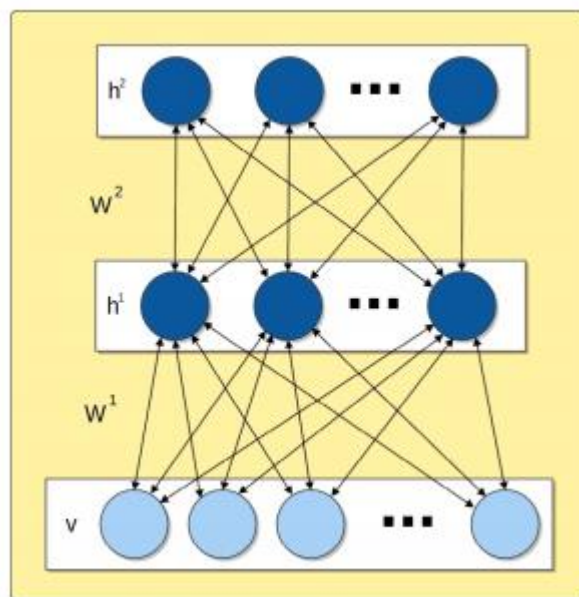
۲-۴-۲. شبکه‌ی باور عمیق

شبکه‌ی باور عمیق که به اختصار با DBN نمایش داده می‌شود توسط هینتون و همکارانش ارائه شد [48]. این شبکه‌ها معماری‌های عمیقی هستند که از مجموعه ماشین‌های بولتزمن محدودشده‌ی انباشته تشکیل شده‌اند و با استفاده از الگوریتم یادگیری ارائه شده در بخش ۲.۴.۱ آموزش داده می‌شوند. این شبکه‌ها، توزیع احتمال توأم را بر روی داده‌های قابل مشاهده و برچسب‌ها بدست می‌آورند و از استراتژی یادگیری حریصانه‌ی لایه‌به‌لایه‌ی بهینه، به منظور مقداردهی اولیه‌ی پارامترهای شبکه عمیق استفاده می‌کند و در نهایت تمامی وزن‌ها توسط خروجی‌های مورد انتظار به‌طور دقیقی تنظیم می‌شوند.

رویه یادگیری حریصانه دو مزیت دارد [51]: (۱) مقداردهی اولیه مناسبی را برای شبکه ایجاد می‌کند و با اینکار، مشکلاتی که در انتخاب پارامتر وجود دارد و ممکن است بهینه‌ی محلی ضعیفی را انتخاب کند تا اندازه‌ای پاسخ می‌دهد. (۲) فرآیند یادگیری بدون سرپرست بوده و به هیچ برچسبی برای دسته‌بندی نیاز ندارد. بنابراین ضرورت داده‌ی برچسب‌دار برای آموزش را از میان برمی‌دارد. ایجاد شبکه‌ی باور عمیق از لحاظ محاسباتی بسیار پرهزینه‌ای است چراکه نیازمند آموزش چندین ماشین بولتزمن محدودشونده است و مشخص هم نیست که چگونه می‌توان حداکثر احتمال آموزش برای بهینه‌سازی مدل را تقریب زد [48].

۳-۴-۲. ماشین بولتزمن عمیق

ماشین بولتزمن عمیق که به اختصار با DBM نمایش داده می‌شود توسط هیلتون و همکارانش ارائه شد [49]. در مقایسه با DBN ها که دو لایه بالایی، مدل گراف بی‌جهت و لایه‌های پایینی، مدل مولد جهت دار را تشکیل می‌دهند DBM دارای اتصالات بدون جهت در سراسر ساختار خود است. همچنین DBM ها دارای چندین لایه از واحدهای پنهان هستند که واحدهای موجود در لایه‌های فرد از واحدهای زوج مستقل هستند و عکس این موضوع نیز صادق است. شکل شماره ۷ معماری یک DBM را نشان می‌دهد [50]:



شکل ۷: معماری DBM

تابع انرژی DBM با دو لایه پنهان، که h^1 و h^2 به ترتیب مخفف لایه‌های پنهان اول و دوم بوده و v که مخفف لایه مشخص است می‌تواند به صورت زیر محاسبه شود [50]:

$$E(v, h^1, h^2) = - \sum_{i=1}^{m^1} \sum_{j=1}^{n^1} v_i h_j^1 w_{ij}^1 - \sum_{i=1}^{m^2} \sum_{j=1}^{n^2} h_i^1 h_j^2 w_{ij}^2 \quad (12)$$

در رابطه‌ی بالا m^1 و m^2 به ترتیب مخفف تعداد واحدهای مشخص در لایه‌های اول و دوم، و n^1 و n^2 به ترتیب مخفف تعداد واحدهای پنهان در لایه‌های اول و دوم هستند. علاوه بر آن ماتریس‌های وزنی $W_{m^1 \times n^1}^1$ و $W_{m^2 \times n^2}^2$ به ترتیب وزن‌های ارتباطی بین بردارهای v و h^1 و بردارهای h^1 و h^2 را رمزگذاری می‌کند [50].

محاسبه‌ی مشتق مدل‌های مبتنی بر RBM بسیار پرهزینه هستند. برای حل این مشکل از الگوریتم واگرایی متقابل استفاده می‌شود. بدین صورت که از وضعیت واحدهای پنهان و مشخص نمونه‌برداری می‌شود. بنابراین، احتمالات شرطی در سراسر دو واحد پنهان و مشخص به شرح زیر آورده شده است [50].

$$p(v_i = 1 | \mathbf{h}^1) = \phi \left(\sum_{j=1}^{n^1} w_{ij}^1 h_j^1 \right) \quad (13)$$

$$p(h_z^2 = 1 | \mathbf{h}^1) = \phi \left(\sum_{j=1}^{m^2} w_{jz}^2 h_j^1 \right) \quad (14)$$

$$p(h_j^1 = 1 | v, \mathbf{h}^2) = \phi \left(\sum_{i=1}^{m^1} w_{ij}^1 v_i + \sum_{z=1}^{n^2} w_{jz}^2 h_z^2 \right) \quad (15)$$

سرانجام مدل تولید شده می‌تواند به صورت زیر نوشته شود [50].

$$p(v) = \sum_{\mathbf{h}^1} P(\mathbf{h}^1) P(v | \mathbf{h}^1) \quad (16)$$

در معادله‌ی بالا $P(\mathbf{h}^1) = \sum_v P(\mathbf{h}^1, v)$ است. علاوه بر این، فرآیند یادگیری RBM دوم ادامه می‌یابد که رابطه‌ی $P(\mathbf{h}^1) = \sum_{\mathbf{h}^2} P(\mathbf{h}^1, \mathbf{h}^2)$ جایگزین می‌شود. با استفاده از چنین روشی، احتمالات شرطی داده شده در معادله‌های ۱۴-۱۲ و واگرایی متقابل می‌تواند پارامترهای DBM را یک بار در یک لایه یاد بگیرد. سپس می‌توان یادگیری مبتنی بر حوزه‌ی میانگین را برای بدست آوردن مدل دقیق‌تر اعمال کرد.

۳. تنظیم پارامترهای شبکه‌ی بولتزمن عمیق با رویکرد فرا ابتکاری

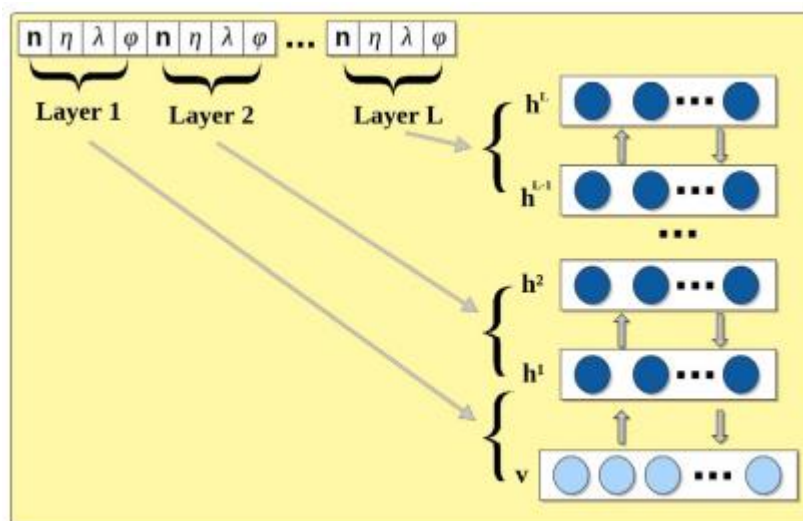
یکی از مسائل بهینه‌سازی که اخیراً مورد توجه محققین قرار گرفته است تنظیم دقیق پارامترهای RBM، DBN و iRBMs با رویکردهای فرا ابتکاری است [13, 14, 50, 52, 53, 54, 55, 56]. چهار پارامتر اصلی که برای حصول نتایج دقیق در ماشین‌های بولتزمن محدودشونده تنظیم می‌شوند عبارتند از: تعداد واحدهای پنهان n ، میزان یادگیری η ، افت وزن λ و مومنتوم ϕ .

ماشین‌های محدودشونده‌ی عمیق، RBM‌ها را به صورت پشته بر روی یکدیگر قرار می‌دهند. اگر یک ماشین محدودشونده‌ی عمیق، L لایه داشته باشد در هر بهینه‌سازی، $4 \times L$ متغیر رمزگذاری می‌شود. همچنین روش آموزش ماشین‌های بولتزمن عمیق روشی حریصانه‌ای است که هر لایه به طور مستقل آموزش می‌بیند و در هر لایه ۴ متغیر بهینه می‌شود.

برای آنکه پارامترهای η ، λ ، n ، ϕ با رویکرد فرا ابتکاری به دقت تنظیم شوند تکنیک‌های بهینه‌سازی به صورت تصادفی مقداردهی اولیه می‌شوند و سپس الگوریتم‌های آن‌ها اجرا می‌شوند. بر اساس نوشته‌های [13, 14, 55, 57] مقادیر پارامترهای ذکر شده با محدوده‌ی زیر پیشنهاد می‌شوند:

$$\eta \in [0.1, 0.9], \quad n \in [5, 100], \quad \phi \in [0.00001, 0.01] \text{ and } \lambda \in [0.1, 0.9]$$

به منظور هدایت عملیات جست‌وجو به بهترین راه‌حل‌ها، از تابع خطای میانگین مربعات* در [50] استفاده شده است. این تابع به اختصار با MSE نمایش داده می‌شود. این تابع با مقایسه‌ی مقدار واقعی و تخمین‌زده شده، میزان خطا را محاسبه می‌کند. شکل شماره‌ی ۸ ایده‌ی کلی مورد استفاده در این کار را برای تنظیم دقیق ابرپارامترهای DBM نشان می‌دهد [50]. با توجه به شکل شماره‌ی ۸، این مسئله‌ی بهینه‌سازی، مجموعه‌ای از ابرپارامترها را انتخاب می‌کند که با توجه به ورودی مدل، میانگین خطای مربعات را در سراسر مجموعه آموزشی به حداقل برساند.



شکل ۸: رویکرد پیشنهادی برای بهینه‌سازی متغیرهای DBM

این مسئله‌ی بهینه‌سازی، مجموعه‌ای از ابر پارامترها را انتخاب می‌کند که با توجه به ورودی مدل، MSE را در سراسر مجموعه آموزشی به حداقل می‌رساند.

۳-۱. الگوریتم‌های فرا ابتکاری مورد استفاده برای تنظیم دقیق پارامترها

جستجوی هارمونی (HI): این الگوریتم در سال ۲۰۰۱ میلادی توسط گیم ارائه شد [58]. این روش از فرآیند نواختن همزمان گروه ارکستر موزیک الهام گرفته است. در این روش فرا ابتکاری تلاش برای به‌دست آوردن هماهنگی در فرآیند موزیک، مشابه پیدا کردن حل بهینه در مسائل بهینه‌سازی است. جستجوی هارمونی مانند الگوریتم ژنتیک، از جمله الگوریتم‌های بهبود دهنده است. به این صورت که با نسلی از بردارهای حل شروع شده و برای ایجاد نسل‌های جدید از فرایند انتخاب استفاده می‌شود. اما برخلاف الگوریتم ژنتیک که در آن از دو کروموزوم برای تولید کروموزوم یا بردار حل جدید استفاده می‌شود در این روش از همه بردارهای حل موجود در حافظه برای تولید حل جدید استفاده می‌شود.

یکی از مشکلات جستجوی هارمونی، زمانی ایجاد می‌شود که مسئله دارای بهینه‌های محلی باشد. در این هنگام ممکن است جستجوی هارمونی در بهینگی محلی متوقف شود و امکان دسترسی به بهینه‌های سراسری را از بین ببرد. دلیل این مشکل عدم کارایی مناسب الگوریتم در اجرای جستجوی محلی در مسائل بهینه‌سازی گسسته است [58]. به‌منظور رفع این مشکل و تطبیق روش حل با مسئله، محققان با تغییر در پارامترها و عملگرهای الگوریتم، انواع مختلفی از این الگوریتم ارائه کردند تا دقت حل و نرخ همگرایی افزایش یابد. PAR و HMCR دو پارامتری هستند که به ترتیب احتمال در نظرگیری متغیر از حافظه و احتمال تغییر متغیر انتخاب شده از حافظه ماتریس به اندازه‌ی تصادفی را نشان می‌دهند که نقش اساسی

* Means Square Error

در تنظیم صحیح بردارهای حل جدید دارند. در مطالعات [13], [53], [54] و [56] از این جستجو برای بهبود ماشین‌های RBM استفاده می‌شود.

جستجوی هارمونی بهبود یافته (IHS): این جستجو، مدل بهبود یافته‌ای از جستجوی هارمونی است. در این رویکرد با در نظر گرفتن نرخ HMRC، از مقادیر پوپا برای هر دو حافظه‌ی هارمونی استفاده می‌کند که مسئول ایجاد راه‌حل‌های جدید بر مبنای آزمایش‌های قبلی پخش‌کننده‌ی موسیقی است و نرخ PAR وظیفه دارد برخی اختلالات را در راه‌حل‌های ایجاد شده توسط HMRC ایجاد کند تا مانع بهینه‌های محلی شود. در هر تکرار پارامترهای HMRC و PAR با مقادیر جدیدی که در محدوده‌ی $[HMCR_{min}, HMCR_{max}]$ و $[PAR_{min}, PAR_{max}]$ است به‌روزرسانی می‌شوند. متغیر بندویس* q برای محاسبات PAR استفاده می‌شود و مقادیر آن بین $[q_{min}, q_{max}]$ است. در مطالعات [50] و [53] از این الگوریتم برای تنظیم پارامترهای DBM و DBN استفاده کرده‌اند.

AIWPSO: این مدل نوعی از الگوریتم‌های ازدحام جمعیت (PSO) است که هر راه حل احتمالی را به عنوان یک ذره (عامل) ازدحام در نظر می‌گیرد. هر عامل دارای یک بردار موقعیت و سرعت در فضای جستجو و همچنین دو ثابت شتاب $c1$ و $c2$ است. به هر عامل مقدار مناسب با هر موقعیت انتساب داده می‌شود و پس از چند بار تکرار، بهترین موقعیت سراسری به عنوان بهترین راه‌حل انتخاب می‌شود. AIWPSO برای ایجاد تعادل بین قابلیت‌های اکتشاف سراسری و محلی PSO پیشنهاد شده است. در هر تکرار، هر عامل با تنظیم پویای w ، وزن اینرسی مناسب را در فضای جستجو انتخاب می‌کند. پاسوس و همکارانش [50] از این الگوریتم برای تنظیم پارامترهای RBM استفاده کرده‌اند.

CS: جستجوی فاخته [59, 60] از ترکیب پرواز زنجیره مارکوف، همراه با رفتار انگلی برخی از گونه‌های فاخته تعریف شده است. این مدل سه ایده‌ی اساسی را دنبال می‌کند:

- ۱- هر فاخته یک بار در لانه‌هایی که به طور تصادفی انتخاب می‌شوند تخم می‌گذارد.
- ۲- پرنده میزبان تخم مرغ فاخته را با احتمال $[0, 1]$ کشف کرده و تخم مرغ را دور می‌اندازد یا لانه را رها کرده و لانه‌ی جدیدی ایجاد می‌کند (راه حل جدیدی ایجاد شود).
- ۳- لانه‌هایی با بهترین تخم مرغ به نسل‌های بعدی منتقل می‌شوند.

در مطالعات [50] و [61] از این الگوریتم برای تنظیم پارامترهای RBM استفاده کرده‌اند

FA: این الگوریتم از جذابیت چشمک‌زن کرم شب‌تاب هنگام جفت‌گیری و جذب طعمه‌های بالقوه حاصل می‌شود. در واقع، جذابیت β کرم شب‌تاب، از طریق موقعیت آن با کرم‌های شب‌تاب دیگری که درون جمعیت هستند محاسبه می‌شود. همچنین روشنایی کرم شب‌تاب با توجه به مقدار تابع هدف در آن موقعیت تعیین می‌شود. علاوه بر این، جذابیت به ضریب جذب نور کرم شب‌تاب γ نیز بستگی دارد. α ضریب آشفستگی تصادفی است که مانع از بهینه‌سازی‌های محلی سیستم می‌شود به این‌صورت که کرم شب‌تاب، پیاده‌روی تصادفی را در فضای جستجو انجام می‌دهد. در مطالعات [50] و [14] از این الگوریتم برای تنظیم پارامترهای DBM و DBN استفاده کرده‌اند.

BSA: این مدل، الگوریتم تکاملی ساده، موثر و سریع است و برای مقابله با مسائلی توسعه یافته است که با محاسبه‌ی کند و حساسیت بیش از حد به پارامترهای کنترلی مشخص می‌شوند. به طور خلاصه، عملیات تقاطع و جهش همراه با انتخاب تصادفی، حافظه‌های ذخیره شده را به کار می‌گیرد تا جمعیت جدیدی از افراد را بر اساس تجربیات گذشته تولید کند. BSA به یک انتخاب مناسب از دو پارامتر نیاز دارد: میزان اختلاط (mix_rate)، که تعداد عناصر افرادی را که در جمعیت جهش

* bandwidth

می‌یابند کنترل می‌کند و همچنین پارامتر F ، که دامنه ماتریس مسیر جستجو را کنترل می‌کند. پاسوس و همکارانش [50] از این الگوریتم برای تنظیم پارامترهای DBM و DBN استفاده کرده‌اند.

JADE: الگوریتم مبتنی بر تکامل افتراقی است که استراتژی جهش "DE/current-to-p-best" را اجرا می‌کند و فقط عوامل p -best را در فرآیند جهش به کار می‌گیرد. علاوه بر این، JADE از آرشیو اختیاری، برای اطلاعات تاریخی و همچنین به روزرسانی تطبیقی در پارامتر کنترلی استفاده می‌کند. JADE نیاز به انتخاب پارامتر C دارد که مخفف میزان سازگاری پارامتر است و g (حرص و طمع) که حرص استراتژی جهش را تعیین می‌کند [50]. پاسوس و همکارانش [50] از این الگوریتم برای تنظیم پارامترهای DBM و DBN استفاده کرده‌اند.

CoBiDE: تکنیک مبتنی بر تکامل افتراقی است که برای نمایش بهتر مختصات سیستم در فرآیند تقاطع، از ماتریس کوواریانس استفاده می‌کند. علاوه بر این، جهش و تقاطع با استفاده از یک توزیع دو بعدی کنترل می‌شود تا مبادله خوبی بین اکتشاف و بهره برداری حاصل شود. احتمال اجرای تکامل دیفرانسیل با توجه به ماتریس کوواریانس با پارامتر p_b تعریف می‌شود، همچنین نسبت افرادی که از جمعیت فعلی برای محاسبه ماتریس کوواریانس انتخاب شده‌اند با ps نشان داده می‌شود [50]. پاسوس و همکارانش [50] از این الگوریتم برای تنظیم پارامترهای DBM و DBN استفاده کرده‌اند.

۲-۳. بررسی کارایی الگوریتم‌های فرا ابتکاری

با توجه به بررسی انجام شده در زمینه‌ی « تنظیم دقیق پارامترهای ماشین‌های RBM » مشخص شد پاسوس و همکارانش [50] جامع‌ترین بررسی را نسبت به سایر مقالات موجود در این زمینه [13, 14, 50, 52, 53, 54, 55, 56] انجام داده‌اند. آن‌ها، مدل‌های DBM و DBN یک لایه، دو لایه و سه لایه را با استفاده از الگوریتم‌های واگرایی متقابل (CD) و واگرایی متقابل پایدار (PCD) مقایسه کرده‌اند. برای بهبود پارامترهای « نرخ یادگیری، تعداد واحدهای پنهان، زوال وزن و مومنتوم » از هفت روش بهینه سازی زیر استفاده شده است که عبارتند از: جستجوی هارمونیک بهبود یافته (IHS)، الگوریتم بهینه‌سازی ازدحام ذرات وزن دار ساکن تطبیقی (AIWPSO)، جستجوی فاخته (CS)، الگوریتم کرم شب تاب (FA)، الگوریتم بهینه‌سازی جستجوی بازگشتی (BSA)، الگوریتم تکاملی دیفرانسیل تطبیقی (JADE) و تکامل دیفرانسیل بر اساس یادگیری ماتریس کوواریانس و الگوریتم تنظیم پارامترهای توزیع دو حالت (CoBiDE).

با مقایسه‌ی عملکرد الگوریتم‌های بالا، پاسوس و همکارانش [50] به نتایج زیر دست پیدا کرده‌اند:

- ۱- بهینه سازی مبتنی بر تکنیک‌های فرا ابتکاری، نتایج دقیق تری را نسبت به جستجوهای تصادفی فراهم می‌کند همانگونه که این موضوع در آثار Papa و همکارانش مشهود است [13, 53, 54]
- ۲- به نظر می‌رسد DBM نتایج دقیق تری را نسبت به DBN ایجاد می‌کنند.
- ۳- هنگامی که پارامترها به طور دقیق تنظیم می‌شوند به نظر نمی‌رسد که تعداد لایه‌ها در نتایج تأثیرگذار باشند.
- ۴- IHS در تمام مجموعه‌های داده در هر دو مدل DBN و DBN بهترین نتایج را بدست آورد، اما نتایج از لحاظ آماری مشابه سایر تکنیک‌های فرا ابتکاری نیز است.
- ۵- در این آزمایش‌ها تفاوت قابل توجهی بین الگوریتم‌های واگرایی انقباضی و تداخلی وجود ندارد، زیرا ۱۰ تکرار برای یادگیری استفاده می‌شود. در واقع انتظار می‌رود الگوریتم واگرایی متقابل پایدار به قیمت یک فرآیند همگرایی طولانی‌تر عملکرد بهتری داشته باشد.

همچنین با توجه به نتایج بدست آمده به نظر می‌رسد DBM ها ممکن است مدل‌های بهتری را نسبت به DBN یاد بگیرند اما تاکید مقاله‌ی [50] بر این است که تکنیک‌های فرا ابتکاری برای تنظیم دقیق پارامترهای DBM مناسب هستند.

۴. نتیجه‌گیری

یادگیری ماشین رویکرد استقرایی است که از داده‌های آموزش دیده برای تولید مدل استفاده می‌کند. این مدل برای تشخیص تصویر، تشخیص گفتار، پردازش زبان طبیعی و ... مفید است. شبکه‌ی عصبی یکی از مدل‌های یادگیری ماشین است که به طور گسترده به عنوان یک مدل یادگیری ماشین مورد استفاده قرار می‌گیرند. شبکه‌های عصبی دارای مدل‌های تک لایه و چند لایه هستند. برای آموزش شبکه‌های عصبی تک لایه از قانون دلتا استفاده می‌شود که این قانون برای حل مسائل خطی مناسب است. اما برای حل مسائل غیر خطی لازم است از شبکه‌های عصبی چند لایه استفاده شود. برای آموزش شبکه‌های عصبی چند لایه از الگوریتم پس‌انتشار استفاده می‌شود. برای طبقه‌بندی شبکه‌ی عصبی، انتخاب تعداد گره‌های خروجی و تابع فعال‌ساز معمولاً به دسته‌بندی باینری (دو کلاس) یا طبقه‌بندی چند کلاسه (سه یا بیشتر) بستگی دارد. یادگیری عمیق به طور ساده یک تکنیک یادگیری ماشین است که از شبکه‌های عصبی عمیق استفاده می‌کند. آموزش لایه‌های پنهان عمیق تر سخت بود و باعث افت عملکرد آن می‌شد. یادگیری عمیق این مشکل را حل کرد. یادگیری عمیق سه نقطه ضعف شبکه‌ی عصبی عمیق را پوشش داد. این سه نقطه ضعف عبارتند از: محوشدگی گرادیان، بیش‌برازش و بار محاسباتی.

تکنیک‌های یادگیری عمیق به چهار دسته‌ی شبکه‌های عصبی پیچشی، ماشین‌های رمزگذار، خودرمزگذارهای تنک و ماشین‌های بولترمن تقسیم می‌شوند. شبکه‌های عصبی پیچشی تصویر ورودی را دریافت می‌کنند و به هر ویژگی تصویر، میزان اهمیتی (وزن‌های قابل یادگیری و بایاس) تخصیص می‌دهند که آن‌ها را قادر به متمایزسازی از یکدیگر می‌کند. ماشین‌های رمزگذار برای رمزگذاری بهینه‌ی عملیات یادگیری مورد استفاده قرار می‌گیرند. خودرمزگذارهای تنک نوع دیگری از انواع کدگذاری‌های عصبی است که برحسب موضع و محل قرارگیری نورون‌ها در برابر کدگذاری متراکم قرار دارد. ماشین‌های بولترمن محدودشونده مدل‌های احتمالی هستند که برای مدل‌سازی توزیع داده‌های ورودی (لایه پنهان) از یک لایه باینری پنهان استفاده می‌کنند که به آن‌ها واحدهای پنهان نیز می‌گویند.

یکی از محدودیت‌هایی که در شبکه‌های عصبی وجود دارد تنظیم تعداد زیادی از پارامترها است که می‌تواند به یک مسئله‌ی غیرقابل حل تبدیل شود. این مقاله مسئله‌ی «تنظیم دقیق پارامترهای ماشین‌های بولترمن محدود شونده با الگوریتم‌های فرا ابتکاری» را به عنوان یک مسئله‌ی بهینه‌سازی مطرح می‌کند و الگوریتم‌های فرا ابتکاری استفاده شده در این زمینه را مورد بررسی قرار می‌دهد. مزیت استفاده از این تکنیک‌ها آن است که به محاسبه‌ی مشتق صدها پارامتر نیاز ندارد البته این راه حل برای فضاهایی با ابعاد بالا توصیه نمی‌شود.

مراجع

- [1] P. Kim, "Machine Learning," in *MATLAB Deep Learning*, 2017, pp. 17-18.
- [2] P. Kim, "Neural Network," in *MATLAB Deep Learning*, 2017, pp. 51-52.
- [3] P. Kim, "Training of Multi-Layer Neural Network," in *MATLAB Deep Learning*, 2017, pp. 79-80.
- [4] P. kim, "Neural Network and Classification," in *MATLAB Deep Learning*, 2017, pp. 101-102.
- [5] P. kim, "Deep Learning," in *MATLAB Deep Learning*, 2017, pp. 120-121.
- [6] b. L. Cun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard and L. D. Jackel, "Handwritten Digit Recognition with a Back-Propagation Network," *Advances in Neural Information Processing Systems*, vol. 2, 1990.
- [7] G. E. Hinton and R. R. Salakhutdinov, "Reducing the Dimensionality of Data with Neural Networks," *Science*, vol. 313, no. 5786, pp. 504-507, 2006.
- [8] J. Zhang, S. Shan, M. Kan and X. Chen, "Coarse-to-Fine Auto-Encoder Networks (CFAN) for Real-Time Face Alignment," *Computer Vision-ECCV 2014: Springer*, pp. 1-16, 2014.
- [9] Y. Zhou, D. Arpit, I. Nwogu and V. Govindaraju, "Is Joint Training Better for Deep Auto-Encoders?," *arXiv:1405.1380*, 2015.
- [10] X. Kong, G. Mao, Q. Wang, H. Ma and W. Yang, "A multi-ensemble method based on deep auto-encoders for fault," *Measurement*, vol. 151, 2019.
- [11] G. E. Hinton, "Training products of experts by minimizing contrastive divergence," *Neural Comput*, pp. 1771-1800, 2002.
- [12] L. A. P. Junior and J. P. Papa, "On the Training Algorithms for Restricted Boltzmann Machine-Based Models Ph.D. thesis," 2018.
- [13] J. P. Papa, G. H. Rosa, K. A. P. D. Costa, N. A. Marana, W. J. Scheirer and D. D. Cox, "On the Model Selection of Bernoulli Restricted Boltzmann Machines Through Harmony Search," *Proceedings of the Companion Publication of the 2015 Annual Conference on Genetic and Evolutionary Computation*, pp. 1449-1450, 2015.
- [14] R. G., P. J., C. K., P. L., P. C. and Y. XS, "Learning Parameters in Deep Belief Networks Through Firefly Algorithm," *Cham: Springer International Publishing*, vol. 9896, pp. 138-149, 2016.
- [15] L. A. Passos and J. P. Papa, "Fine-Tuning Infinity Restricted Boltzmann Machines," *2017 30th SIBGRAPI Conference on Graphics, Patterns and Images (SIBGRAPI)*, p. 63-70, 2017.
- [16] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2818-2826, 2016.
- [17] K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770-778, 2016.
- [18] A. Krizhevsky, I. Sutskever and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Advances in neural information processing systems*, pp. 1097-1105, 2017.
- [19] A. Krizhevsky, I. Sutskever and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Proceedings of the 25th International Conference on Neural Information Processing Systems*, vol. 1, p. 1097-1105, 2012.
- [20] C.-Y. Liou, W.-C. Cheng, J.-W. Liou and D.-R. Liou, "Autoencoder for words," *Neurocomputing*, vol. 139, pp. 1-11, 2014.

- [21] H. Bourlard and Y. Kamp, "Auto-association by multilayer perceptrons and singular value decomposition," *Biological Cybernetics*, vol. 59, pp. 291-294, 1998.
- [22] Y. Zhang, W. Zhang and X. Xiao, "A novel sparse auto-encoder for deep unsupervised learning," *6th Int. Conf. Adv. Comput. Intell. ICACI 2013 - Proc.*, pp. 256-261, 2013.
- [23] H. Lee, C. Ekanadham and A. Ng, "Sparse deep belief net model for visual area V2," *Advances in Neural Information Processing Systems 20*, p. 873-880, 2008.
- [24] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio and P.-A. Manzagol, "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion Pierre-Antoine Manzagol," *Machine Learning Research*, vol. 11, pp. 3371-3408, 2010.
- [25] P. Vincent, H. Larochelle, Y. Bengio and P. A. Manzagol, "Extracting and composing robust features with denoising autoencoders," *Proceedings of the 25th international conference on Machine learning*, pp. 1096-1103, 2008.
- [26] Y. Bengio, A. Courville and P. Vincent, "Representation Learning: A Review and New Perspectives," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, pp. 1798 - 1828, 2013.
- [27] R. Raina, A. Battle, H. Lee, B. Packer and A. Ng, "Self-taught learning: transfer learning from unlabeled data," *Proceedings of the 24th international conference on Machine learning*, p. 759-766, 2007.
- [28] L.-c. L. C. f. i. classification, "Jinjun Wang; Jianchao Yang; Kai Yu; Fengjun Lv; Thomas Huang," *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2010.
- [29] J. Yang, K. Yu, Y. Gong and T. Huang, "Linear spatial pyramid matching using sparse coding for image classification," *IEEE Conference on Computer Vision and Pattern Recognition*, 2009.
- [30] S. Lazebnik, C. Schmid and J. Ponce, "Beyond Bags of Features: Spatial Pyramid Matching for Recognizing Natural Scene Categories," *IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2006.
- [31] J. Wang, J. Yang, K. Yu, F. Lv, T. Huang and Y. Gong, "Locality-constrained Linear Coding for image classification," *IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2010.
- [32] A. Coates and A. Ng, "The importance of encoding versus training with sparse coding and vector quantization," *Proceedings of the 28th International Conference on Machine Learning (ICML-11)*, 2011.
- [33] G. S, T. IW-H, C. L-T and Z. P, "Local features are not lonely-Laplacian sparse coding for image classification," *Computer Vision and Pattern Recognition (CVPR)*, 2010.
- [34] S. Gao, I. W.-H. Tsang and L.-T. Chia, "Laplacian Sparse Coding, Hypergraph Laplacian Sparse Coding, and Applications," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 1, pp. 92-104, 2013.
- [35] K. Yu, T. Zhang and Y. Gong, "Nonlinear Learning using Local Coordinate Coding," *Nips*, p. 2223-2231, 2009.
- [36] J. Wang, J. Yang, K. Yu, F. Lv and T. Huang, "Locality-constrained Linear Coding for image classification," *IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2010.
- [37] X. Zhou, K. Yu, T. Zhang and T. S. Huang, "Image Classification Using Super-Vector Coding of Local Image Descriptors," *Computer Vision-ECCV 2010*, pp. 141-154, 2010.
- [38] Y. Lin, F. Lv, S. Zhu, M. Yang and T. Cour, "Large-scale image classification: Fast feature extraction and SVM training," *CVPR 2011*, 2011.

- [39] K. Balasubramanian, K. Yu and G. Lebanon, "Smooth Sparse Coding via Marginal Regression for Learning Sparse Representations," *arXiv preprint arXiv:12101121*, 20112.
- [40] Y. He, K. Kavukcuoglu, Y. Wang, A. Szlam and Y. Qi, "Unsupervised Feature Learning by Deep Sparse Coding," *Proceedings of the 2014 SIAM International Conference on Data Mining*, 2014.
- [41] G. E. Hinton and T. J. Sejnowski, "Learning and relearning in Boltzmann machines," *Parallel distributed processing: explorations in the microstructure of cognition*, 1986, p. 282–317.
- [42] D. E. A. C. J. B. Y. B. A. e. H. Larochelle, "An empirical evaluation of deep architectures on problems with many factors of variation," *Proceedings of the 24th international conference on Machine learning*, pp. 437-480, 2007.
- [43] R. Salakhutdinov and G. Hinton, "Semantic hashing," *International Journal of Approximate Reasoning*, vol. 50, pp. 969-978, 2009.
- [44] U. Fiore, F. Palmieri, A. Castiglione and A. Santis, "Network anomaly detection with the restricted Boltzmann machine," *Neurocomputing*, vol. 122, pp. 13-23, 2013.
- [45] L. A. d. Silva, K. A. P. d. Costa, P. B. Ribeiro, G. H. d. Rosa and J. P. Papa, "Learning spam features using restricted boltzmann machines," *IADIS International Journal on Computer Science and Information Systems*, vol. 11, pp. 99-114, 2016.
- [46] L. A. Passos, L. A. S. Jr, R. Mendelc, A. Ebigbo, A. Probst, H. Messmann, C. Palm and J. PauloPapa, "Barrett's esophagus analysis using infinity Restricted Boltzmann Machines," *Journal of Visual Communication and Image Representation*, vol. 59, pp. 475-485, 2019.
- [47] P. Khojasteh, L. Aparecido, P. Junior, T. Carvalho, E. Rezende, B. Aliahmad, J. P. Papae and D. KantKumar, "Exudate detection in fundus images using deeply-learnable features," *Computers in Biology and Medicine*, vol. 104, pp. 62-69, 2019.
- [48] G. Hinton, S. Osindero and Y. Teh, "A fast learning algorithm for deep belief nets," *Neural Comput*, p. 1527–1554, 2006.
- [49] R. Salakhutdinov and G. Hinton, "Deep boltzmann machines," *AISTATS*, vol. 1, 2009.
- [50] L. A. Passos and J. P. Papa, "A metaheuristic-driven approach to fine-tune Deep Boltzmann Machines," *Soft Computing Journal*, vol. 97, 2020.
- [51] I. Arel, D. C. Rose and T. P. Karnowski, "Deep Machine Learning - A New Frontier in Artificial Intelligence Research," *IEEE Computational Intelligence Magazine*, vol. 5, no. 4, pp. 13 - 18, 2010.
- [52] L. A. Passos, D. R. Rodrigues and J. P. Papa, "Fine Tuning Deep Boltzmann Machines Through Meta-Heuristic Approaches," *2018 IEEE 12th International Symposium on Applied Computational Intelligence and Informatics (SACI)*, 2018.
- [53] J. P. Papa, G. H. Rosa, A. N. Marana, W. Scheirer and D. D. Cox, "Model selection for Discriminative Restricted Boltzmann Machines through meta-heuristic techniques," *Computational Science*, vol. 9, pp. 14-18, 2015.
- [54] J. P. Papa, W. Scheirer and D. D. Cox, "Fine-tuning Deep Belief Networks using Harmony Search," *Applied Soft Computing*, vol. 46, pp. 875-885, 2016.
- [55] X.-S. Yang and J. P. Papa, "Bio-Inspired Computation and Applications in Image Processing," in *Fine-tuning deep belief networks using cuckoo search*, 2016, pp. 47-59.
- [56] J. P. Papa, A. B. D. R. Pereira and X.-S. Yang, "On the Harmony Search Using Quaternions," *Cham: Springer International Publishing*, pp. 126-137, 2016.
- [57] J. P. Papa, G. H. Rosa, D. R. Pereira and Xin-She Yangb, "Quaternion-based Deep Belief Networks fine-tuning," *Soft Computing*, vol. 60, pp. 328-335, 2017.

- [58] I. Rastgar and R. Sahraeian, "Developing harmony search algorithm for solving optimization problems: a case study in parallel machine production scheduling problem," *Industrial Engineering Research in Production Systems*, vol. 1, pp. 57-71, 2013.
- [59] X.-S. Yang and S. Deb, "Cuckoo Search via Lévy flights," *2009 World Congress on Nature & Biologically Inspired Computing (NaBIC)*, pp. 210-214, 2009.
- [60] N. Nguyen, P. Milanfar and G. Golub, "A computationally efficient superresolution image reconstruction algorithm," *IEEE Transactions on Image Processing*, vol. 10, no. 4, p. 573–583, 2001.
- [61] D. Rodrigues*, X.-S. Yang and J. Papa†, "Fine-tuning deep belief networks using cuckoo search," *International Journal of Mathematical Modelling and Numerical Optimisation*, vol. 1, pp. 330-343, 2010.